

A scalable state-space Python package for spatio-temporal geostatistics

Jacopo Rodeschini ¹, Lorenzo Tedesco ¹

¹University of Bergamo, Dept. of Economics

MFEM Community Workshop
Georgia Tech's School of Computational Science and Engineering

September 22-25, 2026

Table of Contents

- 1 Geostatistics, an overview
- 2 Computational approximation & SPDE main ideas
- 3 State-Space models in spatio-temporal modelling
- 4 Low-Rank State-Space models
- 5 Case study: Air quality in Italy
- 6 The geossm Python package

Geostatistics: overview

Over the last decade, spatial and spatio-temporal analysis has gained increasing relevance (e.g., in **climate science**, **earth observation** and **remote sensing**) because of data availability.

- **Temporal dependence** → phenomena evolve in such a way that past observations influence future ones,
- **Spatial dependence** → nearby locations show similar behaviour.

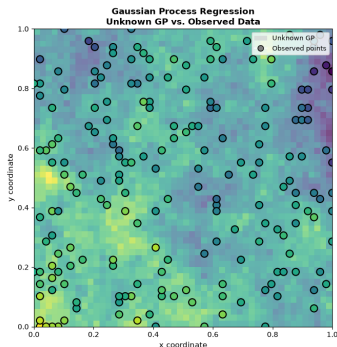
Spatial correlation

$\text{Cov}(y(\mathbf{s}), y(\mathbf{s}'))$ decays as $\|\mathbf{s} - \mathbf{s}'\|$ grows.

Gaussian process regression

An important class of models for **spatial processes** is based on Gaussian processes (GP; Rasmussen, 2003).

- For every finite set $\{\mathbf{s}_1, \dots, \mathbf{s}_n\} \subset \mathcal{D}$, the vector $(y(\mathbf{s}_1), \dots, y(\mathbf{s}_n))^T$ is jointly Gaussian,
- We assume there is an “unknown” latent GP $y(\mathbf{s}) \sim \text{GP}(\mu(\cdot), C(\cdot, \cdot))$, with mean $\mu(\cdot)$ and covariance $C(\cdot, \cdot)$,
- the observed data $\mathbf{y}_{1:n} = (y(\mathbf{s}_1), \dots, y(\mathbf{s}_n))^T$ are a finite realisation of that process.

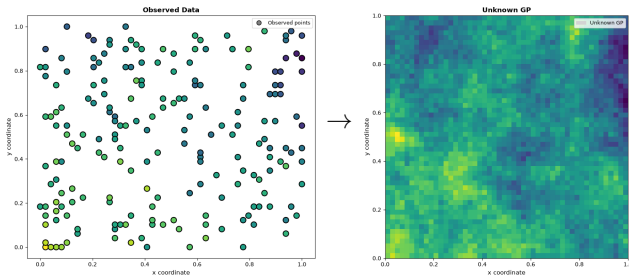


Gaussian process regression

- $\mathbf{y}_{1:n} = (y(\mathbf{s}_1), \dots, y(\mathbf{s}_n))^T \sim N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, with $\Sigma_{ij} = C(\mathbf{s}_i, \mathbf{s}_j)$

Spatial statistics

Given $\mathbf{y}_{1:n}$, is it possible to recover $y(\mathbf{s})$ at an arbitrary, unobserved $\mathbf{s} \in \mathcal{D}$?



Two assumptions on C :

- C is parametrised by $\boldsymbol{\theta}$: $C(\mathbf{s}_i, \mathbf{s}_j; \boldsymbol{\theta})$,
- C is *stationary and isotropic*: $C(\mathbf{s}_i, \mathbf{s}_j; \boldsymbol{\theta}) = C(\|\mathbf{s}_i - \mathbf{s}_j\|_2; \boldsymbol{\theta})$.

Gaussian process regression: bottleneck

- Since $\mathbf{y}_{1:n} \sim N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ and $y(\cdot) \sim \text{GP}(\boldsymbol{\mu}, C)$,
- then, jointly, $(y(\mathbf{s}), \mathbf{y}_{1:n}^\top)^\top \sim N\left(\begin{bmatrix} \mu(\mathbf{s}) \\ \boldsymbol{\mu} \end{bmatrix}, \begin{bmatrix} \sigma^2(\mathbf{s}) & \boldsymbol{\sigma}(\mathbf{s})^\top \\ \boldsymbol{\sigma}(\mathbf{s}) & \boldsymbol{\Sigma} \end{bmatrix}\right)$, with $\boldsymbol{\sigma}(\mathbf{s}) = (C(\mathbf{s}, \mathbf{s}_1), \dots, C(\mathbf{s}, \mathbf{s}_n))^\top$.

Spatial regression (kriging)

$y(\mathbf{s}) \mid \mathbf{y}_{1:n} \sim N(\mu_c(\mathbf{s}), \Sigma_c(\mathbf{s}))$, with

- $\mu_c(\mathbf{s}) = \boldsymbol{\mu}(\mathbf{s}) + \boldsymbol{\sigma}(\mathbf{s})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{y}_{1:n} - \boldsymbol{\mu})$
- $\Sigma_c(\mathbf{s}) = \sigma^2(\mathbf{s}) - \boldsymbol{\sigma}(\mathbf{s})^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\sigma}(\mathbf{s})$

(with measurement error $\epsilon \sim N(0, \sigma_\epsilon^2)$, replace $\boldsymbol{\Sigma}$ by $\boldsymbol{\Sigma} + \sigma_\epsilon^2 \mathbf{I}_n$)

- Pro: we have a closed-form solution,
- Con: the Cholesky factorisation of $\boldsymbol{\Sigma}$ costs $\mathcal{O}(n^3)$ time and $\mathcal{O}(n^2)$ memory,

which quickly becomes prohibitive as n increases (e.g remote sensing, $n = 10^5$, storing $\boldsymbol{\Sigma}$ requires 80 GB in double precision).

Gaussian process regression: simulation

To check the computational demand:

- $\mathcal{D} = [0, 1] \times [0, 1]$,
- Simulate $\mathbf{y}_{1:n}$ for $n \in \{100^2, 150^2, \dots, 300^2\}$ from a known $\text{GP}(\cdot; \boldsymbol{\theta})$,
- Compute the runtime for $\mu_c(\mathbf{s})$ and $\Sigma_c(\mathbf{s})$.

n	Runtime (s)
10000	8
22500	42
40000	164
62500	522
90000	1510

Table 1: Runtime of the closed-form kriging predictor as a function of n , averaged over 5 simulations (CPU backend).

- Empirical growth $\approx n^{2.4}$ (optimised BLAS)
- As a result, the “full” GP regression is not feasible when n becomes large.

Computational approximation

- To deal with the big- n problem, several approximation methods have been proposed in the literature (Heaton et al., 2019).
- One approach relies on the **stochastic partial differential equation (SPDE)** representation of GPs (Lindgren et al., 2011),
- In particular, on $\mathbb{R}^d$ a stationary GP $u(\mathbf{s})$ with Matérn covariance is the (weak) solution of

$$(\kappa^2 - \Delta)^{\alpha/2} u(\mathbf{s}) = \mathcal{W}(\mathbf{s}), \quad \mathbf{s} \in \mathcal{D} \subset \mathbb{R}^d, \quad \kappa > 0, \quad (1)$$

where Δ is the Laplacian, $\mathcal{W}(\mathbf{s})$ is spatial white noise, κ controls the spatial range, $\alpha = \nu + d/2$, and $\nu > 0$ controls the smoothness of the field.

SPDE solution

To obtain a numerical solution of Eq. (1),

- ① Given a discretised domain $\mathcal{G}_h = (\mathcal{V}, \mathcal{E})$, and $R = |\mathcal{V}|$,
- ② $u(\mathbf{s})$ is approximated by its finite element (FE) representation

$$u(\mathbf{s}) = \sum_{i=1}^R \psi_i(\mathbf{s}) w_i = \boldsymbol{\psi}(\mathbf{s})^\top \mathbf{w},$$

where $\{\psi_i\}_{i=1}^R$ are piecewise-linear basis functions on $\mathcal{G}_h$, and $\mathbf{w} = (w_1, \dots, w_R)^\top$ are Gaussian weights.

- ③ Formulate the stochastic weak formulation of Eq. (1):

$$\left\langle \phi_j, (\kappa^2 - \Delta)^{\alpha/2} x \right\rangle \stackrel{d}{=} \langle \phi_j, \mathcal{W} \rangle, \quad \forall j = 1, \dots, m,$$

for suitable test functions $\{\phi_j\}_{j=1}^m$, where

$$\langle f, g \rangle = \int_{\mathcal{D}} f(\mathbf{s}) g(\mathbf{s}) d\mathbf{s}.$$

SPDE solution

- ① Substitute the FE representation into the weak formulation;
- ② Considering the case $\alpha = 2$, Galerkin choice $\phi_j = \psi_j$
- ③ Apply Green's first identity with homogeneous Neumann boundary conditions:

$$(\kappa^2 \mathbf{C} + \mathbf{G}) \mathbf{w} \stackrel{d}{=} \mathbf{b}, \quad \mathbf{b} \sim N(\mathbf{0}, \mathbf{C})$$

whit

$$C_{jk} = \langle \psi_j, \psi_k \rangle, \quad (2)$$

$$G_{jk} = \langle \nabla \psi_j, \nabla \psi_k \rangle; \quad (3)$$

- ④ It follows that

$$\mathbf{w} \sim N(\mathbf{0}, \mathbf{K}_{\kappa}^{-1} \mathbf{C} \mathbf{K}_{\kappa}^{-1}), \quad \mathbf{K}_{\kappa} = \kappa^2 \mathbf{C} + \mathbf{G},$$

or, equivalently, the precision matrix of $\mathbf{w}$ is

$$\mathbf{Q}_{\kappa} = \mathbf{K}_{\kappa} \mathbf{C}^{-1} \mathbf{K}_{\kappa}. \quad (4)$$

The SPDE solution using (py)MFEM

Eqs. (2)–(3) are the mass and stiffness **bilinear forms**

- 1 Build a mesh and an H^1 finite element space:

```
import mfem.ser as mfem
fec = mfem.H1_FECollection(1, mesh.Dimension()) #linear FE (psi_i)
fespace = mfem.FiniteElementSpace(mesh, fec)
```

- 2 Assemble $\mathbf{C}$ via MassIntegrator:

```
C_form = mfem.BilinearForm(fespace)
C_form.AddDomainIntegrator(mfem.MassIntegrator()) # <psi_j, psi_k>
C_form.Assemble(); C_form.Finalize()
C = C_form.SpMat()
```

- 3 and $\mathbf{G}$ via DiffusionIntegrator;

```
G_form = mfem.BilinearForm(fespace)
G_form.AddDomainIntegrator(mfem.DiffusionIntegrator()) # <grad psi_i,
G_form.Assemble(); G_form.Finalize()
G = G_form.SpMat()
```

The SPDE solution using (py)MFEM

Note that:

- $\mathbf{C}$ is not diagonal so that $\mathbf{Q}_{\kappa}$ is *not* sparse,
- Replace $\mathbf{C}$ by the approximation $\mathbf{C} = \langle \psi_j, 1 \rangle$ (Lindgren et al., 2011),

Simulation results:

With observations $\mathbf{y}_{1:n} = \mathbf{A}\mathbf{w} + \epsilon$, with $A_{ij} = \psi_j(\mathbf{s}_i)$, $\epsilon \sim N(\mathbf{0}, \sigma_\epsilon^2 \mathbf{I}_n)$,

n	RMSE	Runtime GP(s)	Runtime SPDE (s)	Speedup
10000	0.30	8	0.87	9.20×
22500	0.32	42	1.83	22.95×
40000	0.34	164	3.93	41.73×
62500	0.35	522	7.83	66.67×
90000	0.36	1510	14.42	104.72×

Table 2: Runtime/accuracy of the SPDE-based ($R = 400$) vs. the full GP, as a function of n averaged over 5 simulations (CPU backend).

The role of time

Now let's consider a replication of the spatial process $y(\mathbf{s})$ over time:

- $y(\mathbf{s}) \rightarrow y_t(\mathbf{s})$ at discrete time $t \in \{1, \dots, T\}$,
- Processes are often correlated in time (e.g. today's air pollution is linked to yesterday's),
- meaning $\text{Cov}(y_t(\mathbf{s}), y_{t'}(\mathbf{s})) \neq 0$ for $t \neq t'$.

Given the full observation set $\{\mathbf{y}_{1,1:n}, \dots, \mathbf{y}_{T,1:n}\}$:

Research question

Can we leverage the temporal correlation to improve prediction of the spatial process at time t , $y_t(\mathbf{s})$?

Idea: embed the sparse SPDE precision of Eq. (4) into a dynamical model which captures the temporal evolution of the process.

State-Space models in spatio-temporal modelling

We consider the State-Space Models (SSMs; Shumway et al., 2000), which provide a flexible framework for dynamic spatio-temporal regression.

- Dynamic formulation:
 - **Measurement equation**: explains the observed variability,
 - **State equation**: models the (latent) system dynamics.

Intuition:

- *Time*: let the vertex weights evolve as $\mathbf{w}_t = \mathbf{F}\mathbf{w}_{t-1} + \boldsymbol{\eta}_t$ by following a Markov process;
- *Space*: $\boldsymbol{\eta}_t \sim N(\mathbf{0}, \mathbf{Q}_\kappa^{-1})$ encodes the Matérn-type spatial correlation across mesh vertices;

Let the FE weights, $\mathbf{w}_t$, at the mesh vertices be the latent state, and drive their evolution in time t using a spatial innovation $\boldsymbol{\eta}_t$:

$$\boldsymbol{\eta}_t \sim N(\mathbf{0}, \mathbf{Q}_\kappa^{-1}), \quad \text{with } \mathbf{Q}_\kappa \text{ exactly as in Eq. (4).}$$

Low-Rank State-Space model (LR-SSM)

Given $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and the finite element space $\mathcal{F}$, the proposed univariate (multivariate) LR-SSM is defined as:

$$y_t^R(\mathbf{s}) = \mathbf{X}_t(\mathbf{s})^\top \boldsymbol{\beta} + \mathbf{W} \boldsymbol{\Phi}^\top(\mathbf{s}) \mathbf{w}_t + \epsilon_t(\mathbf{s}), \quad (5)$$

$$\mathbf{w}_t = \mathbf{F} \mathbf{w}_{t-1} + \boldsymbol{\eta}_t, \quad \boldsymbol{\eta}_t \sim N(\mathbf{0}, \mathbf{Q}_\kappa^{-1}), \quad (6)$$

where:

- $\mathbf{w}_t = (w_t(\mathbf{v}_1), \dots, w_t(\mathbf{v}_R))^\top$, with $\mathbf{v}_i \in \mathcal{V}$,
- ϵ_t and $\boldsymbol{\eta}_t$ are mutually independent and i.i.d. over t .
- $\mathbf{W}$ is a loading matrix mapping the (q -variate) latent field to the (p -variate) observed process,
- $\boldsymbol{\Phi}(\mathbf{s}) \mathbf{w}_t$ is the low-rank representation of the continuous latent spatial process $\mathbf{w}_t(\mathbf{s})$, such that

$$\boldsymbol{\Phi}(\mathbf{s}) \mathbf{w}_t \longrightarrow \mathbf{w}_t(\mathbf{s}) \quad \text{as the mesh size } h \rightarrow 0. \quad (7)$$

The model is parametrised by

$$\Pi = \{\boldsymbol{\beta}, \mathbf{W}, \mathbf{F}, \sigma_\epsilon^2, \boldsymbol{\kappa}, \boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0\},$$

which must be estimated from the observed data;

Low-Rank State-Space model (LR-SSM): inference

- Π is estimated by maximum likelihood via the **EM algorithm**,
- (E-step) given Π , the moments of $\mathbf{w}_t \mid \mathbf{y}_{1:T}$ are computed by the Kalman filter and the Kalman smoother,
- Since $\mathbf{w}_t \in \mathbb{R}^R$ with $R \ll n$, the cost per iteration is $\mathcal{O}(R^3 T)$,
- Standard errors for $\hat{\Pi}$ come from the observed Fisher information, computed by **automatic differentiation** (JAX) of the log-likelihood $\ell(\Pi)$:

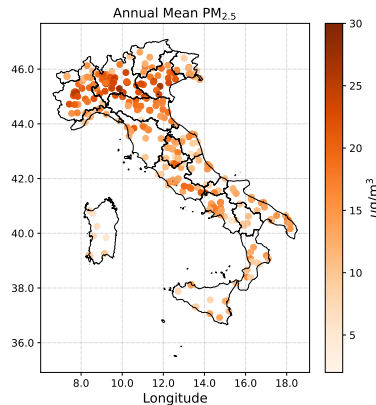
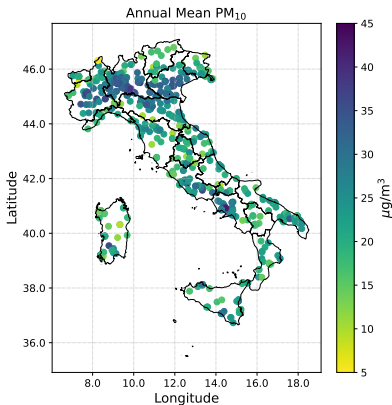
$$\widehat{\text{Var}}(\hat{\Pi}) \approx \left(- \frac{\partial^2 \ell(\Pi)}{\partial \Pi \partial \Pi^\top} \Big|_{\Pi = \hat{\Pi}} \right)^{-1}.$$

Role of the vertices R

- The FE approximation error decreases as the mesh is refined ($h \rightarrow 0$, i.e. R increases) (Rodeschini et al., 2026);
- As expected, a finer FE mesh gives increased inference accuracy, at a higher cost.

Case study: Air quality in Italy

- We applied the LR-SSM to (PM_{10} , $\text{PM}_{2.5}$) data in Italy (mid-2018 to 2023), i.e. $T = 2008$ days and $n = 797$,
- PM_{10} and $\text{PM}_{2.5}$ are observed at partially different stations, heterotopic settings, and are known to be highly correlated ($\simeq 0.95$).



Case study: Air quality in Italy, validation

The LR-SSM was applied to study the PM_{10} and $PM_{2.5}$ concentrations ($\mu g/m^3$) across Italy.

- We use 10% of random stations per pollutant for validation, with the remaining 90% for training.
- We consider low-rank approximations where $LR = \frac{R}{n}$ is set to 100%, 75%, 50%, and 25%.

LR	R	y	Train		Validation		Time
			RMSE	R ²	RMSE	R ²	
100%	516	PM_{10}	3.52	0.95	6.88	0.81	3h 37m
	281	$PM_{2.5}$	2.24	0.96	5.44	0.80	
75%	387	PM_{10}	4.26	0.92	6.87	0.81	1h 51m
	211	$PM_{2.5}$	2.94	0.93	5.47	0.80	
50%	258	PM_{10}	5.00	0.89	6.44	0.83	1h 00m
	141	$PM_{2.5}$	3.51	0.90	5.62	0.79	
25%	129	PM_{10}	6.07	0.84	7.31	0.78	16m
	71	$PM_{2.5}$	4.43	0.84	6.26	0.74	

Case study: Air quality in Italy

- Joint **multivariate modelling** (under a heterotopic sampling scheme).
- Naturally handles **missing observations** (i.e. 28.7% and 34.1%) without pre-processing.

Predictions at unobserved locations: given the smoothed moments $(\hat{\mathbf{w}}_{t|T}, \mathbf{P}_{t|T})$ they cost $\mathcal{O}(RT)$ complexity.

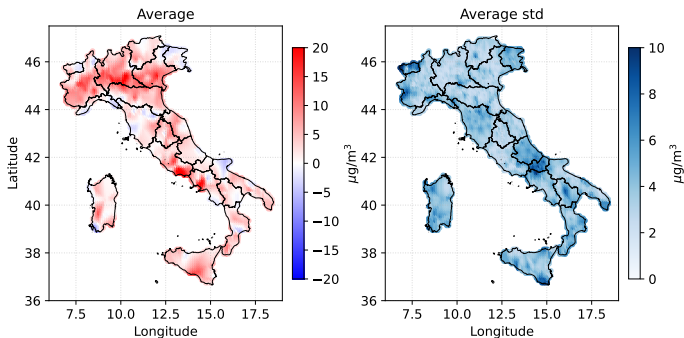
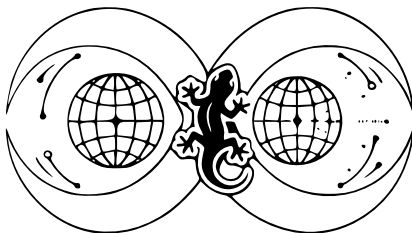


Figure 1: PM₁₀ random effect component $\mathbf{W}\Phi(\mathbf{s})\mathbf{w}_t$ average

The geossm Python package: a turning point

We make LR-SSMs easy to use with the geossm Python package.



geossm = **Ge**ostatistics with the **SSM** framework

```
pip install geossm
```

- geossm works directly with `geopandas.GeoDataFrame` objects.

- **Modular Pipeline:**

- Spatio-temporal dataset handling
- Covariance models (based on SPDEs)
- State-space models
- Spatio-temporal state-space models

```
geossm.dataset  
geossm.covmodel  
geossm.ssm  
geossm.stmodel
```

The geossm Package

Key Features

- Native integration with `geopandas.GeoDataFrame` objects,
- Mesh objects created via the **gmsh** API (and used with `pyMFEM` to assemble the stiff and the mass matrix as in Eqs. (2)–(3)),
- Efficient inference via the EM algorithm, with likelihood and its derivatives implemented in **Python/JAX**, enabling automatic differentiation and scalable execution on multi-core CPUs,
- Native support for **GPU and TPU acceleration**.

Within the SSM framework, `geossm` provides a flexible environment for developing and extending geostatistical models.

- **Contributions are welcome!**
- Star the **GitHub** repository if you find it useful.

The geossm package: the LR-SSM

```
from geossm.covmodel import spdeAppoxCov as matern_spde
from geossm.stmodel import LRStateSpaceModel as lrssm

dataframe, domain = ... # Import the dataset
mesh_io = ...          # Build mesh via gmsh API

# Create the Matern SPDE obj.
cov_fun = matern_spde(domain, rescale=1)
cov_fun = cov_fun.setup(mesh_io) # Set the mesh

# Build the model
model = lrssm(
    dataframe, # GeoDataframe
    ['AQ_pm10 ~ 1 + WE_temp_2m', ...], # Measurement eqs.
    verbose=True, backend='cpu', # Backend: cpu / gpu / tpu
    domain = [domain]) # Convex domain

model = model.setup([cov_fun]) # Set the cov. object

# Fit the model
results = model.fit()
```

Future development: manifolds and metric graphs

Processes on **manifolds** and **graphs** are increasingly relevant in spatial and spatio-temporal statistics.

Idea: keep the LR-SSM structure and change the domain of the SPDE.

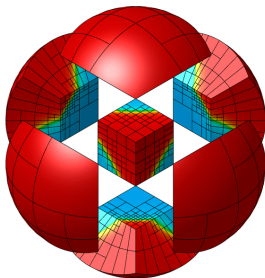
- **Manifolds:** formulate the SPDE directly on the manifold, e.g. the sphere $\mathbb{S}^2$ (Lindgren et al., [2011](#)).
- **Metric graphs** $\mathcal{G} = (\mathcal{V}, \mathcal{E})$:
 - *Covariance-based:* isotropic covariances in the resistance metric are valid only for restricted classes (e.g. Matérn with $\nu \leq 1/2$) (Porcu et al., [2023](#)).
 - *SPDE-based:* $(\kappa^2 - \Delta_{\mathcal{G}})^{\alpha/2} u = \mathcal{W}$ on $\mathcal{G}$, with Kirchhoff vertex conditions (Bolin et al., [2024](#)) .

A scalable state-space Python package for spatio-temporal geostatistics

Thank you for your attention!

Any questions?

jacopo.rodeshini@unibg.it



My question

The hardest part of my FE pipeline was not the SPDE... it was the mesh.

Given

- observation sites $\{\mathbf{s}_i\}_{i=1}^n \subset D \subset \mathbb{R}^d$, with it's intensity estimate
- a vertex budget R

Find a triangulation $\mathcal{G}_h$ with R vertices such that

- *data-driven*: more nodes where there are more data (intensity function).
- *shape-regular*: the transition from small triangle to high triangles is smooth.
- *consistent*: $h_{\max} \rightarrow 0$ as $R \rightarrow \infty$.

Is there a Python library that does this?

(C++ with Python interface is also accepted.)

References

-  Bolin, David, Alexandre B Simas, and Jonas Wallin (2024). “Gaussian Whittle–Matérn fields on metric graphs”. In: *Bernoulli* 30.2, pp. 1611–1639.
-  Heaton, Matthew J et al. (2019). “A case study competition among methods for analyzing large spatial data”. In: *Journal of Agricultural, Biological and Environmental Statistics* 24, pp. 398–425.
-  Lindgren, Finn, Håvard Rue, and Johan Lindström (2011). “An explicit link between Gaussian fields and Gaussian Markov random fields: the stochastic partial differential equation approach”. In: *Journal of the Royal Statistical Society Series B: Statistical Methodology* 73.4, pp. 423–498.
-  Porcu, Emilio, Philip A White, and Marc G Genton (2023). “Stationary nonseparable space–time covariance functions on networks”. In: *Journal of the Royal Statistical Society Series B: Statistical Methodology* 85.5, pp. 1417–1440.
-  Rasmussen, Carl Edward (2003). “Gaussian processes in machine learning”. In: *Summer school on machine learning*. Springer, pp. 63–71.
-  Rodeschini, Jacopo et al. (2026). “Multivariate low-rank state–space model with SPDE approach for high-dimensional data”. In: *Spatial Statistics* 73, p. 100971. ISSN: 2211-6753. DOI: <https://doi.org/10.1016/j.spasta.2026.100971>. URL: <https://www.sciencedirect.com/science/article/pii/S2211675326000199>.
-  Shumway, Robert H, David S Stoffer, and David S Stoffer (2000). *Time series analysis and its applications*. Vol. 3. Springer.